**Quester Tangent Corporation**

**Southeastern Pennsylvania Transportation Authority (SEPTA)**
**Silverliner V**

**Central Diagnostic System (CDS)**

# Car Monitoring Network Interface Control Document

**(Network Interoperability)**


**Prepared by:** **Christian Leclerc**
**Network Integration Engineer**

**Verified by:** **David Gregson**
**Lead Hardware Engineer**

**Verified by:** **Rick Pearson**
**Lead Software Engineer**

**Approved by:** **Jim Kightley**
**Project Engineer**



**Issue Date: February 12, 2008**
**Document Number: DDN-T097-0080**
**Revision: R06**


QUESTER TANGENT™
Making Data Intelligent™

| DATE | REVISION | DESCRIPTION |
|---|---|---|
| 8-Jan-2007 | R00 – Draft | Draft version provided for initial information and discussion. |
| 17-Jan-2007 | R00 | Capture results of meetings with Rotem.<br>General clarifications throughout. |
| 05-Feb-2007 | R01 | 4.8 – Add EMC/EMI compliance specifications.<br>7.1, 7.3.5 – Add File Transfer type "Real Time Data Record" for Propulsion and Braking subsystems.<br>7.3.1 – Clarify software update regarding Neuron processor.<br>9.2, 9.3, 9.4 – Clarify testing phase responsibilities. |
| 13-Apr-2007 | R02 | 3.2 – Added allowance for alternative connector on approval.<br>4.8 – Test standards finalized.<br>5.1 – Additional nodes added.<br>6 – Section reorganized and expanded to describe functional behaviour and define network variables.<br>7 – Section expanded to provide file transfer protocol details.<br>8 – Section renamed and expanded to include details for network node implementation.<br>9 – Clarifications and updates based on subsystem supplier comments. |
| 08-Jun-2007 | R03 | Throughout document: 'nvi', 'nvo', 'nci' removed from network variable names to increase readability.<br>5.3 – Clarify installation location and location error behaviour.<br>6.1.2 – Add section.<br>6.2.3.1 – Describe stopping individual parameter transmissions.<br>6.3.3 – Simplify service configuration table.<br>6.5.3 – Describe stopping individual parameter transmissions.<br>6.5.4 – CDTParameter transmission rate set by CDT.<br>6.5.5 – Delete CDTSubscribe, add CDTStatusBits.<br>6.6.2 – Add Propulsion specific SCI update reason value to table.<br>7.1 – Move file transfer type information to individual ICDs.<br>8.1.1 – Add notes on self-documentation.<br>8.3 – Add notes for Host-based Nodes.<br>8.4.1 – Add self-Installation notes with example code sketch. |
| 17-Aug-2007 | R04 | 1.5 – Clarify document evolution.<br>4.8 – Add label to EMI / EMC standards table.<br>5.1 – Add subnet/node information to node assignment table.<br>6.2.1 – Add clarification of ObjRequest and ObjStatus usage.<br>6.4.2 and 6.4.3 – Remove support for Self-Test, Clear Reset and Reset Object Requests.<br>8.1.2 – Supplier notification of variable declaration updates also applies to Host-based nodes. |
| 16-Nov-2007 | R05 | 6.5.1 – Clarify that StatusBits.bit0 and StatusBits2.bit0 are not used. Data within StatusBits and StatusBits2 starts at bit1.<br>7.3 – Clarify file transfer protocol. |

| DATE | REVISION | DESCRIPTION |
|---|---|---|
| 12-Feb-2008 | R06 | 6.2.3.2 – Clarify the loss of communication condition.<br>6.4.5 – Change requirement for NetConfig support from “shall” to “should”. This feature is recommended but not mandatory.<br>8.1.2 – Remove “Update Net Variable Config” as a requirement for host-based nodes. |

**COPYRIGHT**

Copyright © 2008 by Quester Tangent Corporation. All rights reserved. This item and the information contained herein are the property of Quester Tangent Corporation. No part of this document may be reproduced, transmitted, stored in a retrieval system, or translated into any language or computer language in any form or by any means, electronic, mechanical, magnetic, optical, chemical, manual, or otherwise, without the express written permission of Quester Tangent Corporation.

Quester Tangent Corporation
Marine Technology Centre
201-9865 West Saanich Road
Sidney, B.C., Canada V8L 5Y8
Tel: 250-656-6677
Fax: 250-655-4696

# Table of Contents

1 Introduction ..... 7
1.1 Purpose ..... 7
1.2 Motivation ..... 7
1.3 Scope ..... 7
1.4 Audience ..... 8
1.5 Document Evolution ..... 8
1.6 Definitions and Acronyms ..... 8
1.7 Central Diagnostic System Functional Overview ..... 8
2 References ..... 10
3 Network Configuration ..... 11
3.1 Network Topology ..... 11
3.2 Network Wiring Interface ..... 11
3.3 Network Cable Specifications ..... 11
1.4 Network Termination ..... 11
4 Network Physical Layer Interface Design ..... 12
4.1 Network Processor Selection ..... 12
4.2 LonWorks Free Topology Transceiver ..... 12
4.3 Status Indicators and Service Pin Switch ..... 12
4.4 DC Blocking Capacitor ..... 12
4.5 Common Mode Isolation Choke ..... 12
4.6 Magnetic Shield ..... 12
4.7 Spark Gap Replacement ..... 13
4.8 EMI / EMC Compliance ..... 13
1.9 Network Interface Design Review ..... 13
5 Network Organization ..... 14
5.1 Network Architecture ..... 14
5.2 Network Management ..... 15
5.3 Node Location ..... 15
6 Network Interface Implementation ..... 17
6.1 Network Interface Overview ..... 17
6.1.1 Network Interface Functions ..... 17
6.1.2 Network Variable Naming Conventions ..... 17
6.2 Network Node Functional Behaviour ..... 17
6.2.1 Network Node Management ..... 17
6.2.2 Time Synchronization ..... 18
6.2.3 Status Monitoring ..... 18
6.2.4 SCI Image Update ..... 19
6.2.5 SCI Version Query ..... 20
6.3 Network Variables ..... 20

6.3.1 Generic Network Node ..... 20
6.3.2 General Information ..... 22
6.3.3 Network Variable Services ..... 25
6.4 Node Object Variables ..... 25
6.4.1 TimeStamp ..... 25
6.4.2 ObjRequest ..... 25
6.4.3 ObjStatus ..... 26
6.4.4 FileReq and FileStat (File Transfers) ..... 26
6.4.5 NetConfig Configuration Property ..... 27
6.5 Monitoring Object Variables ..... 27
6.5.1 StatusBits and StatusBits2 ..... 27
6.5.2 Parameter ..... 27
6.5.3 Subscribe ..... 28
6.5.4 CDTParameter ..... 28
6.5.5 CDTStatusBits ..... 28
6.5.6 TransferState ..... 29
6.6 SCI Update Object Variables ..... 29
6.6.1 SCIUpdateReq ..... 29
6.6.2 SCIUpdateResp ..... 30
6.7 SCI Version Object Variables ..... 31
6.7.1 SCIVersionReq ..... 31
6.7.2 SCIVersion ..... 31
7 File Transfers ..... 33
7.1 File Types ..... 33
7.2 File Transfer Protocol ..... 33
7.3 SCI Update Protocol ..... 34
7.3.1 Software Image File Formats ..... 36
8 Interface Rules and Conventions ..... 37
8.1 Node Documentation ..... 37
8.1.1 Neuron-based Nodes ..... 37
8.1.2 Host-based Nodes ..... 37
8.2 Neuron Conventions ..... 38
8.3 Variable Declaration Rules ..... 38
8.3.1 Neuron-based Nodes ..... 38
8.3.2 Host-based Nodes ..... 38
8.4 Network Installation ..... 38
8.4.1 Self Installation ..... 38
8.4.2 Neuron Chip Firmware Version ..... 39
8.5 External Interface File ..... 39
8.6 Additional Features ..... 40

9 Subsystem Commissioning ..... 41
9.1 Subsystem Interface Definition Process ..... 41
9.2 Supplier In-house Testing ..... 41
9.3 Interoperability Testing ..... 41
9.4 Vehicle Integration Testing ..... 42

# List of Tables

Table 1: Definitions ..... 8
Table 2: Reference Documents ..... 10
Table 3: EMI / EMC Standards ..... 13
Table 4: Silverliner V Node Idenfication ..... 15
Table 5: Network Variable Types ..... 23
Table 6: Network Variable Service Configuration ..... 25
Table 7: Object Status Bit Used ..... 26
Table 8: Definition of SCI Update Request Values ..... 30
Table 9: Definition of SCI Update Response Values ..... 30
Table 10: Definition of SCI Update Reason Values ..... 31
Table 11: Definition of SCI_Type Values ..... 32
Table 12: File Types Supported in File Transfer ..... 33
Table 13: Subsystem Names ..... 36

# List of Figures

Figure 1: Single Car Monitoring Network ..... 14
Figure 2: Married-Pair Monitoring Network ..... 14
Figure 3: Generic Network Node ..... 21
Figure 4: Logical File Transfer Mechanism ..... 33
Figure 5: SCI Update File Transfer Protocol ..... 35

# 1 Introduction

## 1.1 Purpose

This document describes the interface and interoperability details for the car monitoring network installed in the Electric Multiple Unit (EMU) Commuter Rail Cars, identified as the "Silverliner V" cars, to be manufactured by Rotem and to be operated by the Railroad Division of the Southeastern Pennsylvania Transportation Authority (SEPTA).

## 1.2 Motivation

The car monitoring network provides the communications infrastructure necessary to support the following car requirements:

**"Systems which are microprocessor-based shall … constantly monitor their own system health and report problems as a fault to the Central Diagnostics Terminal." (TS 15.2)**

**"The CDT shall interface with the wireless LAN for maintenance. … This system shall also allow uploading Automatic Vehicle Location System database updates of location triggers and messages. " (TS 15.3)**

**"Using the CDT, it shall be possible to ascertain the current software version of all systems utilizing a microprocessor." (TS 15.3)**

**"From the CDT, it shall be possible to upload new control software to all vehicle-level systems and subsystems utilizing a microprocessor, " (TS 15.3)**

**" the CDT shall allow updates to the Automatic Vehicle Location System database of location triggers and messages via the serial RS-232 port and over the wireless LAN at each of SEPTA's Regional Railroad yard and shop location in accordance with Section 9.12." (TS 15.3)**

**"All software updates of the [Communications] system database shall be made at the Central Diagnostic Terminal (see Section 15.3), or over the wireless LAN which shall interface with the CDT." (TS 9.9)**

**"The [Communications] system shall include a digital, crystal controlled clock which shall permit periodic synchronization from the Global Positioning System … The GPS obtained time shall be used for all system clocks on the train " (TS 9.9)**

**"Within each car, the [Communications] system shall interface with the onboard Central Diagnostics Terminal (see Section 15.3) to achieve the specified functionality. A gateway apparatus shall interface with both the CTD and wireless bridge that will be cabled to the appropriate antenna assembly offering optimum performance for communicating with the wayside WLAN access points." (TS 9.12)**

This document and the supporting references are intended as guidelines for engineers who are familiar with ANSI C, Neuron® C, the LonTalk protocol, and network management.

The subsystem suppliers are responsible for the implementation of the information presented and referenced in this specification

## 1.3 Scope

This document covers the following items:

- The physical monitoring network configuration for the Silverliner V cars.

- The LonWorks nodes operating on the car monitoring network.
- The network variables used by the LonWorks nodes.
- The hardware and software requirements for LonWorks nodes.

## 1.4 Audience

This document is intended for the following parties:

- Rotem engineers responsible for network wiring on the Silverliner V cars.
- Quester Tangent engineers as a point of reference for the design of the network and CDT.
- Subsystem supplier management and engineering staff for the purposes of design and integration test planning.
- SEPTA Engineering and QA representatives responsible for oversight of design and test activities.

## 1.5 Document Evolution

Earlier draft versions of this document provided the basic information needed by subsystem suppliers to facilitate subsystem concept design. Later updates provided additional details and clarifications. The document content is now complete and will be maintained if any further clarifications or modifications are required.

## 1.6 Definitions and Acronyms

**Table 1: Definitions**

| TERM | DEFINITION |
|---|---|
| Car Monitoring Network | Communications network that allows maintenance technicians access to overall vehicle diagnostic information through the Central Diagnostic Terminal. The network also has capacity for limited point-to-point communication between nodes. |
| Network Integrator | Party responsible for defining interfaces between subsystems on the car monitoring network and for ensuring interoperability within the final vehicle configuration. |
| Subsystem supplier | Supplier of a system or subsystem that has a communications interface to the car monitoring network. |
| Node | A subsystem connected to the car monitoring network. |

See the CDS Acronyms and Abbreviations document, DCM-T097-0011.

## 1.7 Central Diagnostic System Functional Overview

The Central Diagnostic Terminal (CDT) provides a central location for monitoring current status of the subsystems in the same car. The CDT obtains status information from subsystems via the car monitoring network and from the vehicle itself by direct connection to the vehicle wiring. Monitoring of all current status information is available via the display on the front of the CDT or PTU laptop. The CDT can activate a set of fault trainlines that control an array of LEDs on the Fault Display Panel (FDP) in the train operator's cab.

The monitoring scheme is based on a "publish and subscribe" model. Each subsystem makes its data available via its network interface ("publish"), and the CDT determines which data the subsystem should transmit and at what rate ("subscribe").

The CDT logs a subset of status information when a fault condition is detected. Logged information is available to the vehicle maintenance personnel either via a display on the front of the CDT, via download to the PTU laptop or via automated transfer to wayside for archive and processing.

The CDT is loaded with configuration data that determines its monitoring and logging functions. The configuration data itemizes all of the status information for every subsystem monitored by the CDT, including descriptive labels and “help text” that provides the maintainer with information related to fault conditions. Also the configuration data determines which particular data items are stored in the CDT’s log and which items are transferred to wayside.

# 2 References

This document shall be used in conjunction with the following documents and publications. If the referenced documents or publications are superseded by an approved revision, the most recent revision shall apply.

**Table 2: Reference Documents**

| PUBLISHER | DOCUMENT NUMBER | TITLE |
|---|---|---|
| SEPTA | Conformed April 2006 | Silverliner V Electric Multiple Unit Commuter Rail Fleet Procurement Technical Specification |
| Echelon | Latest version | Neuron Chip Data Book |
| | Latest version | LonWorks FTT-10A Free Topology Transceiver User's Guide |
| | Latest version | Neuron C Reference Guide |
| | Latest version | Neuron C Programmer's Guide |
| | Latest version | LonWorks Host Application Programmer's Guide |
| | Latest version | File Transfer (LonWorks Engineering Bulletin) |
| | Latest version | Design Practices for Using a LonWorks Free Topology Channel on Trains |
| LONMARK™ Interoperability Association | Latest version | LONMARK™ Application Layer Interoperability Guidelines |
| | Latest version | SNVT Master List |
| | Latest version | SCPT Master List |
| International Electrotechnical Commission (IEC) | EN 61000-4-2 | Electromagnetic Compatibility (EMC). Part 4: Testing and Measuring Techniques. Section 2: Electrostatic Discharge Immunity Test |
| | EN 61000-4-3 | Electromagnetic Compatibility (EMC). Part 4: Testing and Measuring Techniques. Section 3: Radiated, Radio-Frequency, Electromagnetic Field Immunity Test |
| | EN 61000-4-4 | Electromagnetic Compatibility (EMC). Part 4: Testing and Measuring Techniques. Section 4: Electrical Fast Transient/Burst Immunity Test |
| | EN 61000-4-5 | Electromagnetic Compatibility (EMC). Part 4: Testing and Measuring Techniques. Section 5: Surge Immunity Test |
| | EN 61000-4-6 | Electromagnetic Compatibility (EMC). Part 4: Testing and Measuring Techniques. Section 6: Immunity to Conducted disturbances, induced by radio-frequency fields |
| | 62236-3-2 | Railway Applications – Electromagnetic Compatibility |
| | CISPR 11 | Limits and Methods of Measurement of Radio–Disturbance Characteristics of Industrial, Scientific and Medical (ISM) Radio–Frequency Equipment |
| IEEE | | IEEE Dictionary |
| | IEEE 1473 | Standards for Communications Protocol Aboard Trains |
| EIA | 709.3 | Free Topology Twisted Pair Channel Specification |

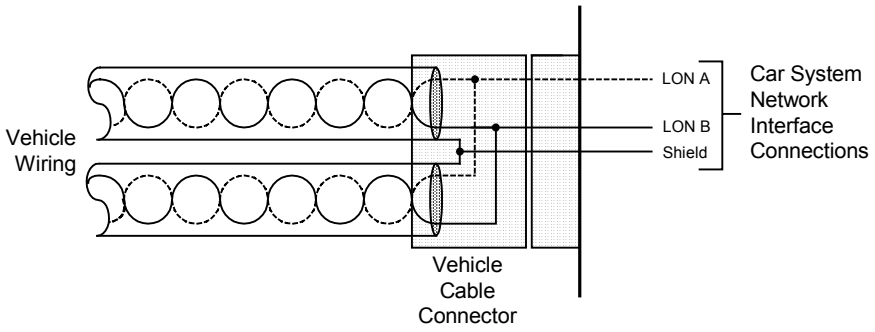
# 3 Network Configuration

## 3.1 Network Topology

The network topology shall be a bus topology as recommended in the Echelon Engineering bulletin "*Design Practices for Using a LonWorks Free Topology Channel on Trains*".

## 3.2 Network Wiring Interface

The network cable shall be continuous and shall not rely on connections within a vehicle system to maintain continuity. This is illustrated in the following sketch.




This configuration maintains the integrity of the network and allows it to continue operating if one or more of the vehicle systems is removed from the vehicle or is disconnected from the network. A subsystem supplier may propose an alternative connector arrangement that achieves these same goals and submit for approval by Rotem and Quester Tangent. Any connector configuration must meet the requirements of the SEPTA Technical Specification.

## 3.3 Network Cable Specifications

The network wiring shall be shielded twisted pair cable with a nominal impedance of 100 ohms as recommended in Echelon's Engineering bulletin "*Design Practices for Using a LonWorks Free Topology Channel on Trains*".

The cable shall meet the following specifications in addition to meeting the requirements of the SEPTA Silverliner V Engineering Specification:

| | |
|---|---|
| Impedance: | 100 ohms ± 10% |
| Attenuation: | ≤ 1 dB / 1000 ft at 100 kHz |
| Propagation Delay: | ≤ 2 nSec / ft at 100 kHz |
| Mutual Capacitance: | ≤ 20 pF / ft at 100 kHz |
| Capacitance Unbalance to Shield: | ≤ 1% at 100 kHz |

## 3.4 Network Termination

The bus termination shall be 100 ohm ± 1 ohm and it shall be DC isolated from the network. The CDT provides a network terminator on its LonWorks communications interface. The CDT also provides a network terminator on another pair of vehicle connector contacts, to provide for termination of the other end of the bus at the CDT. Optionally Rotem may supply equivalent bus termination external to the CDT to avoid returning the cable to the CDT for termination.

The network cable shield is terminated at the CDT and shall not be terminated in any other vehicle systems.

# 4 Network Physical Layer Interface Design

The network shall be implemented using a LonWorks Network conforming to IEEE 1473 (Type L). The network physical layer interface design shall conform to the guidelines presented in Echelon's Engineering Bulletins: *LonWorks FTT-10A Free Topology Transceiver User's Guide* and *Design Practices for Using a LonWorks Free Topology Channel on Trains*.

## 4.1 Network Processor Selection

The network processor may be implemented with an Echelon (www.echelon.com) FT3150 or FT3120 Smart Transceiver or with a "second source" 3150 / 3120 device. Second source devices are available from Toshiba (www.toshiba.com/taec), Cypress (www.cypress.com/control) and others.

## 4.2 LonWorks Free Topology Transceiver

The physical layer interface will be implemented with either an Echelon FT-X1 Communications Transformer or an Echelon FTT-10A Free Topology Transceiver. Refer to Echelon's "*LonWorks FTT-10A Free Topology Transceiver User's Guide*" and "*Design Practices for Using a LonWorks Free Topology Channel on Trains*" for details regarding the implementation of the physical layer interface.

Note: The FT-X1 is only compatible with the Echelon FT3150 / FT3120 devices, the FTT-10A is required if the Toshiba or Cypress devices are used.

## 4.3 Status Indicators and Service Pin Switch

All systems that interface to the network shall be equipped with the following network activity and status indicators:

- Tx: Green LED
- Rx: Green LED
- Service: Yellow LED
- Service Pin: Momentary pushbutton switch

The indicators and push button switch shall be accessible and visible to the maintenance personnel while the equipment is installed and operating in the vehicle.

## 4.4 DC Blocking Capacitor

The network physical layer shall be capable of withstanding a continuous application of a DC voltage of either polarity with a magnitude equal to the maximum possible LVPS voltage. Refer to *the "LonWorks FTT-10A Free Topology Transceiver User's Guide*" and "*Design Practices for Using a LonWorks Free Topology Channel on Trains*" for details regarding AC coupling of a LonWorks network.

## 4.5 Common Mode Isolation Choke

If the FTT-10A Free Topology Transceiver is used a common mode isolation choke as described in "*LonWorks FTT-10A Free Topology Transceiver User's Guide*" and "*Design Practices for Using a LonWorks Free Topology Channel on Trains*" shall be included in the network interface circuitry.

## 4.6 Magnetic Shield

The FT-X1 or FTT-10A shall be equipped with a magnetic shield device as described in "*LonWorks FTT-10A Free Topology Transceiver User's Guide*" and "*Design Practices for Using a LonWorks Free Topology Channel on Trains*".

## 4.7 Spark Gap Replacement

Surge absorbing flash tubes such as the Mitsibushi DSP-140N devices shall be used as recommended in "*Design Practices for Using a LonWorks Free Topology Channel on Trains*".

## 4.8 EMI / EMC Compliance

In addition to any other EMI/EMC compliance requirements that may apply to their equipment, each subsystem supplier shall test and verify compliance with the following standards:

**Table 3: EMI / EMC Standards**

| # | STANDARD | TEST DESCRIPTION | TEST LEVEL | PERFORMANCE CRITERIA | COMMENT |
|---|---|---|---|---|---|
| 1 | IEC 61000-4-2 | Electrostatic Discharge | Level 3 | B | Test both contact and air discharge |
| 2 | IEC 61000-4-3 | Radiated Immunity | Level 3 | A | |
| 3 | IEC 61000-4-4 | Fast Transient Bursts | Level 4 | A | Input / Output Port Level<br>Test as per Section 7.3.2 |
| 4 | IEC 61000-4-5 | Surge Immunity | Level 2 | A | Use test set-up described in Section 7.5 *Shielded Cable*. |
| 5 | IEC 61000-4-6 | Conducted Disturbance | Level 3 | A | Use direct injection to network cable shield as per section 6.2.1 |
| 6 | EIA-709.3 (Note 2) | Conducted Emissions | X | N/A | Emissions on network conductors not to exceed 90 dB/µV, 9 kHz – 150 kHz |

To verify the performance criteria are being met during the testing, the equipment under test (EUT) shall be connected to an external LonWorks test node with the EUT and test node configured to exchange packets on a continuous basis. Confirmation that the performance criteria have been met is defined to be no disruption or degradation in the exchange of packets between the EUT and test node for the duration of the test.

The network cabling used during the testing shall conform to the requirements of section 3.3 and shall be configured as a terminated bus, 10 metres long, with the EUT located at the mid-point of the bus. The test node may be located at either end of the bus or at an intermediate point along it taking into consideration the requirements for Auxiliary Equipment isolation as defined in the above standards.

Test results shall be provided to Rotem and Quester Tangent.

Note: Tests 1–5 in the above table are encompassed by IEC 62236-3-2.
Test 6 ensures that the node complies with EIA 709.3 section 5.3.2 as referenced by IEEE 1473.

Note 2: Test will be performed using CISPR 11 Section 5.1.2 as a reference test method.

## 4.9 Network Interface Design Review

In order to avoid misuse of the Echelon product and also to accelerate the development process, each LonWorks node supplier should submit its network interface design to Rotem for review by Quester Tangent. Quester Tangent will provide the supplier with an assessment of the design and help identify any issues early in the development process. Experience has shown that such design reviews can identify schematic design errors, potential EMI and ESD problems as well as possible power supply and reset problems. Quester Tangent assures strict confidentiality.

# 5 Network Organization

## 5.1 Network Architecture

Each Silverliner V single car has its own complete, independent, stand-alone Lonworks monitoring network. The Silverliner V married-pair has a network that spans the "A" and "B" cars.

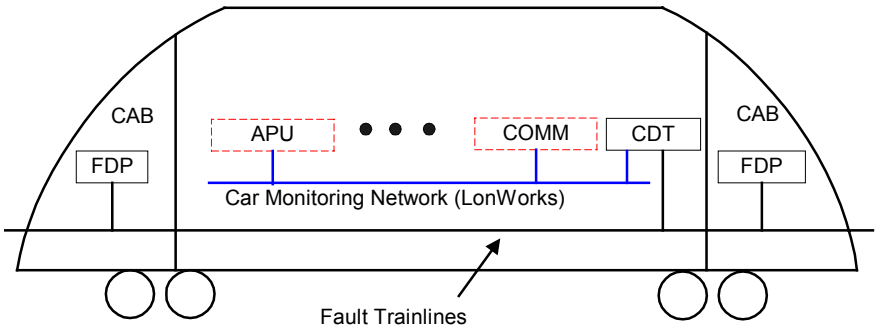



**Figure 1: Single Car Monitoring Network**

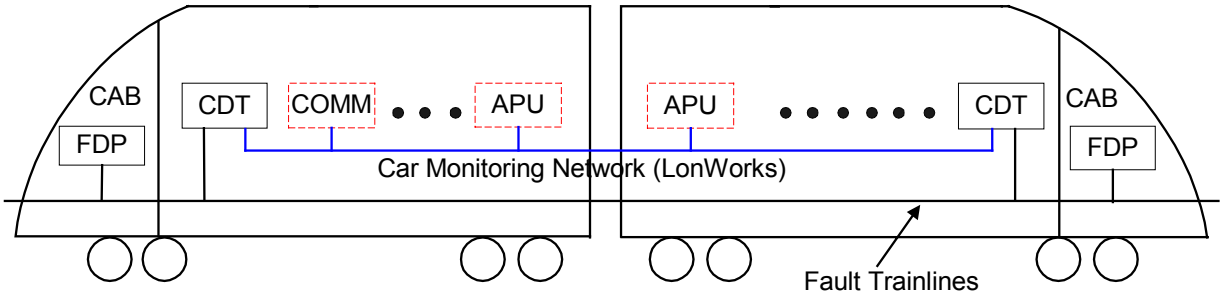



**Figure 2: Married-Pair Monitoring Network**

In a married pair, each CDT monitors only the subsystems in the same car. The connection of the network between the two cars is to provide access to the Communications subsystem, which provides time synchronization and serves as the point of access to wayside for both cars.

The CDT is installed in the electrical equipment locker at the “F” End of each car. The FDP is installed in a cab console panel.

The LonTalk protocol provides for transport of variables from node to node across the network. As part of this mechanism each physical node can be assigned up to two domains, with a logical node address. The Silverliner V car monitoring network shall operate in one domain. Domain index ‘0’ shall be used for all subsystems. This domain shall be zero length.

For subnet/node addressing, the subnet number is 1 for all nodes in a single-car and for the A-car of a married-pair. The subnet number is 2 for all nodes in the B-car of a married-pair. The node numbers are the same in all cars and are defined in Table 4.

**Table 4: Silverliner V Node Idenfication**

| SUBSYSTEM | NODE IDENTIFIER | NODE NUMBER | SUBNET/NODE NUMBER | | |
|---|---|---|---|---|---|
| | | | SINGLE | MARRIED PAIR A | MARRIED PAIR B |
| Central Diagnostic Terminal | CDT | 1 | 1/1 | 1/1 | 2/1 |
| Propulsion System | PROP | 2 | 1/2 | 1/2 | 2/2 |
| Friction Brake System | FBCU | 3 | 1/3 | 1/3 | 2/3 |
| Auxiliary Power System | APU | 4 | 1/4 | 1/4 | 2/4 |
| Heating, Ventilation and Air Conditioning System, A-End (Note 1) | HVAC-A | 5 | 1/5 | 1/5 | 2/5 |
| Heating, Ventilation and Air Conditioning System, F-End (Note 1) | HVAC-F | 10 | 1/10 | 1/10 | 2/10 |
| Cab Signal / ATC System | ATC | 6 | 1/6 | 1/6 | 2/6 |
| Door System (Door Control Relay Panel) | DCRP | 7 | 1/7 | 1/7 | 2/7 |
| Communications System | COMM | 8 | 1/8 | 1/8 | NA |
| Door System (Door Control Units) | DCU1 – DCU6 | 17 – 22 | 1/17 - 1/22 | 1/17 - 1/22 | 2/17 - 2/22 |

Note 1: Refer to SEPTA Technical Specification section 1.14 for definition of A-End and F-End.

For example, the Door system's DCRP in a single-car has subnet/node address 1/7 and the Friction Brake systems in the A-car and B-car of a married-pair have addresses 1/3 and 2/3 respectively.

The number of nodes is 15 (single-car) or 29 (married-pair).

## 5.2 Network Management

All subsystems shall self-install by default, i.e. set the subsystem's own network configuration without intervention by a network management tool. Full details of the required settings will be provided in later versions of this document and/or in individual subsystem ICDs.

All nodes shall support possible future usage of a network management tool facilitated through the NetConfig configuration property (see section 6.4.5).

## 5.3 Node Location

With the exception of the COMM system, all nodes must be able to sense which type of car they are installed in so that the corresponding subnet/node address information can be configured during its boot sequence.

Normally, vehicle equipment is required to be correctly configured automatically as soon as it is installed, i.e. without operator action. One design option is that car wiring provides an open or closed connection between two pins in a subsystem vehicle interface connector ("car strapping") to distinguish the single car and the A-car of a married pair from the B-car of a married pair. See SEPTA Technical Specification section 1.12 "Interchangeability" for further details.

The exact mechanism by which each subsystem identifies its location is outside the scope of this document.

Depending on the mechanism used, a subsystem may be able to determine that the location detection has a failure. The supplier should choose appropriate behaviour of the subsystem in case of a failure. Quester Tangent's recommendation is that the subsystem not communicate at all on the car monitoring network, because it cannot be sure of the correct network address.

# 6 Network Interface Implementation

## 6.1 Network Interface Overview

### 6.1.1 Network Interface Functions

Communication between the CDT and other subsystems attached to the car monitoring network shall adhere to the network interface described in the following sections. The LonTalk protocol uses network variables to communicate data items between nodes on a LonWorks network. Files are sent between nodes using an interoperable file transfer mechanism; the file transfer is managed using network variables but the actual transfer of data is implemented with explicit messages.

The following functions are supported by the interface implementation:

- Network node management
- Time synchronization
- Subsystem status monitoring and data flow management
- SCI image update
- SCI version information query

These functions are mapped to logical groups call Functional Blocks. The Functional Blocks are a collection of network variables used to perform the defined functions.

This generic implementation defines four Functional Blocks:

1. Node Object
2. Monitoring Object
3. SCI Update Object
4. SCI Version Object

Specific network node implementations are described in the individual subsystem ICDs.

### 6.1.2 Network Variable Naming Conventions

Within individual subsystem-specific ICDs, each network variable name has a prefix ‘nvi’ or ‘nvo’, which is a convention used to denote inputs and outputs to a subsystem respectively. Network variable names also contain a 2-digit number associated with a network interface functional block, e.g. nvo01StatusBits. The prefix ‘nci’ is used for non-volatile configuration settings.

For simplicity in the current document, the prefixes and functional block numbers are omitted. This detail affects only the software implementers of CDT and other subsystems.

## 6.2 Network Node Functional Behaviour

### 6.2.1 Network Node Management

Network nodes are designed to use self-installation but provide the facility to permit future installation of the car network using a network management tools if required. General node status and control functions are implemented with standardized network variables. The CDT shall send the ObjRequest network variable to request high-level subsystem status information for network diagnostic purposes. The subsystem shall respond to the CDT ObjRequest network variable by updating and sending its ObjStatus variable.

The following network variables are used for these functions:

- ObjRequest (section 6.4.2)

- ObjStatus (section 6.4.3)
- NetConfig (section 6.4.5)

### 6.2.2 Time Synchronization

The time is defined according to Coordinated Universal Time (UTC), as per the SEPTA Technical Specification. A time stamp network variable shall be broadcast at system startup and each hour when the system is operating. All network nodes shall use this information to adjust their own internal real time clocks to synchronize with train time.

The following network variable is used for this function:

- TimeStamp (section 6.4.1)

### 6.2.3 Status Monitoring

Subsystem status monitoring consists of two types of information; boolean status bits and numeric information. Boolean status network variables are acknowledged (ACKD) by the CDT. Numeric network variables are not acknowledged (UNACKD).

The boolean status bits are concatenated into a single representation with each bit indicating either a fault or an operating status for the subsystem. As a minimum, subsystem suppliers shall provide status bits that indicate the health of the subsystem, and shall also include status bits to indicate the failure of any communication link or major module within the subsystem.

A bit shall have a value of 0 to indicate false, OFF, or alarm clear/reset. A bit shall have a value of 1 to indicate true, ON, or alarm set. Unused bits shall be assigned a value of 0. Each status bit may be used to indicate a failure or an operational status. If a bit indicates a failure then a value of one (1) means failure. For operational statuses, a value of one (1) is used to indicate when a condition is present and zero (0) when the condition is absent.

After a failure or condition causes a bit to be set to one (1), the bit value of zero (0) shall be assigned when: (a) the failure/condition no longer exists; and (b) the bit value of one (1) has been successfully transmitted to the CDT at least once. This is to assure that the CDT receives all fault indications.

Quantitative data (e.g. speed, voltage, current, temperature, etc.) from a node is categorized as a numeric parameter. This information is not concatenated or compressed for transmission to the CDT. Each numeric parameter is transmitted using the same network variable. A numbered identifier assigned in the individual subsystem ICD document is used to differentiate each type of numeric parameter. Each numeric parameter requires a separate transmission of the network variable.

Network bandwidth and CDT processing capability may limit the number of numeric parameters requested for transmission to the CDT. A limited number of quantitative data items may be sent from the CDT to other subsystems.

The following network variables are used for these functions:

- Subscribe (section 6.5.3)
- StatusBits (section 6.5.1)
- Parameter (section 6.5.2)
- CDTParameter (section 6.5.4)
- CDTStatusBits (section 6.5.5)
- TransferState (section 6.5.6)

#### 6.2.3.1 Data Flow Management

Upon subsystem initialization or loss of communication (see section 6.2.3.2), the subsystem shall set its transmission rate shall be once every nine (9) seconds for all boolean status information. All numeric parameter transmissions shall be stopped following initialization or when a loss of communication is detected.

The CDT determines there is an active subsystem on the car monitoring network when it first detects the periodic (every nine seconds) transmission of subsystem boolean status information. The CDT subscribes to subsystem information by sending a new transmission rate for the concatenated boolean information and each of the numeric parameters it wishes to receive.

When the subsystem receives a valid subscription request, it shall start sending the corresponding network variable at the transmission rate requested by the CDT. The first transmission of the network variable shall be within the transmission rate requested. The timer for the first transmission starts when the initial subscription request is received. Each update shall be at the requested transmission rate relative to the preceding transmission. A tolerance of ± 250ms is allowed for the receipt timing of all status information.

When the CDT chooses to stop transmission for a parameter variable, it subscribes to the variable with a zero (0) transmission rate. If the transmission rate for the parameter is already zero (0) then the subsystem shall send the parameter once when the subscribe is received.

When the CDT performs particular file transfers, it may choose to stop all other nodes from transmitting, so that the file transfer will complete as quickly as possible. The CDT achieves this by sending the TransferState network variable at the start of the file transfer and periodically while the file transfer is in progress.

#### 6.2.3.2 Loss of Communications

The CDT uses the receipt of StatusBits network variables to determine loss of communication with each subsystem. Loss of communication is defined as four (4) consecutive missed StatusBits network variable updates.

Likewise, each subsystem shall use the StatusBits network variable's acknowledgement (ACK) to determine loss of communication with the CDT. Loss of communication is defined as not detecting any ACK after four (4) consecutive updates of the StatusBits network variable. Note that this refers to application-level update of the network variable, not the low-level retry transmissions of the packet. The subsystem shall revert to the default transmission rates (boolean status every nine seconds, numeric parameters not sent) when loss of communication is detected. It is recommended that loss of communications with the CDT be incorporated within the subsystem's internal diagnostics and logged in the subsystem's internal fault log, if available.

### 6.2.4 SCI Image Update

The Silverliner V Technical Specification requires, "From the CDT, it shall be possible to upload new control software to all vehicle-level systems and subsystems utilizing a microprocessor, " (TS 15.3)

The CDT uses the LonWorks file transfer protocol (see section 7) to support the upload process. Details of SCI image updates are described in section 7.3.

The following network variables are used for these functions:

- FileReq (section 7.2)
- FileStat (section 7.2)
- SCIUpdateReq (section 6.6.1)
- SCIUpdateResp (section 6.6.2)

### 6.2.5 SCI Version Query

The CDT will query the subsystem for Software Configuration Item (SCI) version information by sending a network variable containing an SCI index number. SCI index numbers are documented in the individual subsytem ICDs. The subsystem shall populate the SCI version information network variable with version information from the requested SCI and transmit the variable to the CDT.

To enumurate all SCIs associated with a particular subsystem, the CDT will make repeated SCI queries. The CDT will increase the SCI index number with each query until the subsystem indicates the CDT has requested an invalid index.

The following network variables are used for these functions:

- SCIVersionReq (section 6.7.1)
- SCIVersion (section 6.7.2)

## 6.3 Network Variables

The car monitoring network uses both LonWorks Standard Network Variable Types (SNVT) and User-Defined Network Variable Types (UNVT). Details about variables used by all subsystems are described in the following sections. Network variables that are unique to a particular subsystem are documented in the individual subsystem ICD.

### 6.3.1 Generic Network Node

Figure 3 shows a schematic of the network interface for a generic network node. Interface features used by specific subsystems are defined in the individual subsystem ICD.

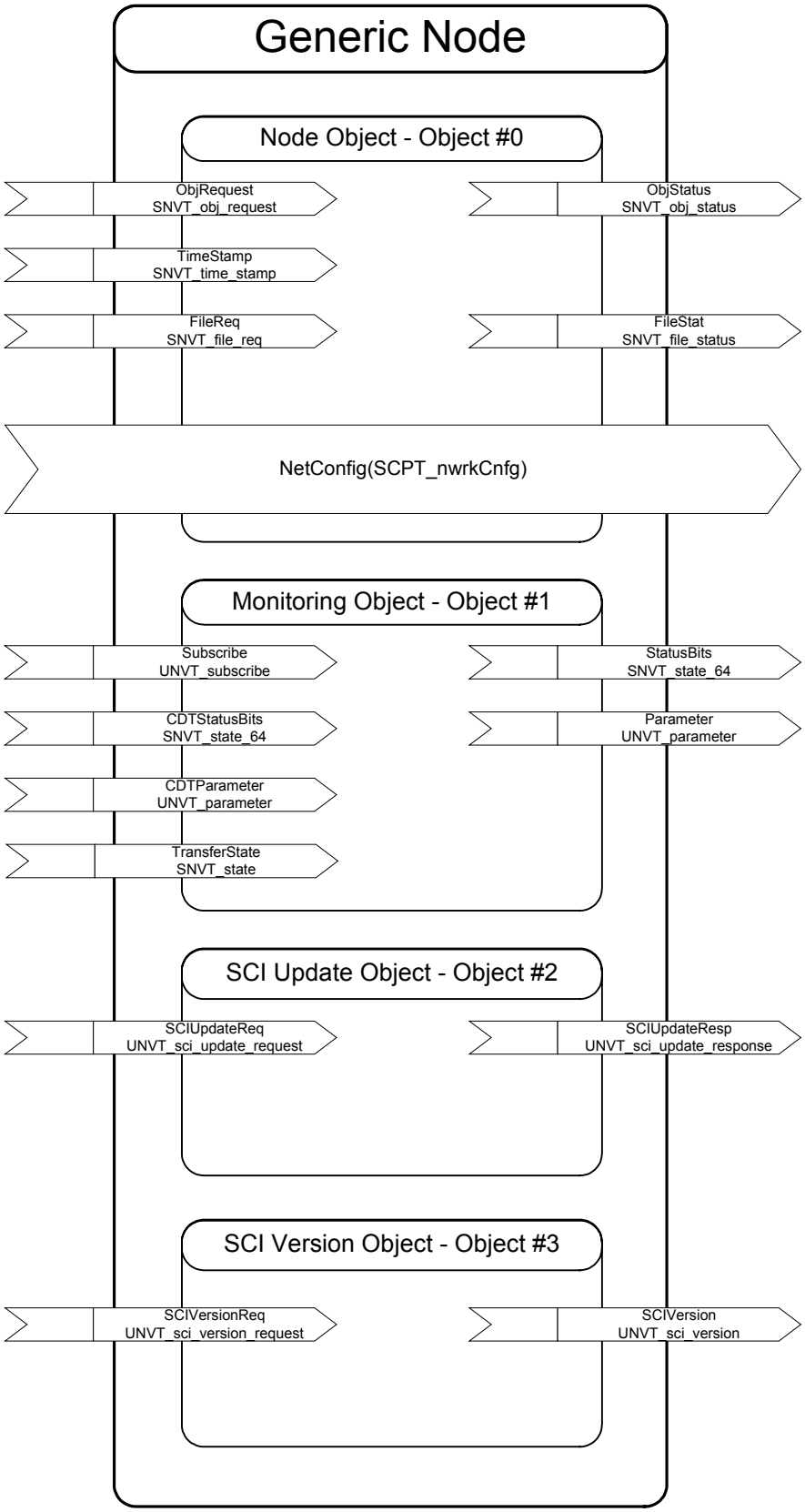



**Figure 3: Generic Network Node**

### 6.3.2 General Information

Network variables provide the mechanism for the control and transport of diagnostic information across the network.

This section describes a generic implementation independent of specific data provided by each subsystem to the CDT. The generic implementation uses standard data types (SNVTs) where practical, and defines custom data types (UNVTs) where needed for efficiency or clarity.

**Table 5: Network Variable Types**

| Name | Source Node | Destination Node | Size (bytes) | Type | Default Period (sec) | Normal Period (sec) | Notes |
|---|---|---|---|---|---|---|---|
| ObjRequest | CDT | All except CDT | 3 | SNVT_obj_request | Async | Async | Mandatory network variable for implementing LonMark Functional Profile: Node Object (SFPTnodeObject). Provides Node self-test and reset requests. |
| ObjStatus | All except CDT | CDT | 6 | SNVT_obj_status | Async | Async | Mandatory network variable for implementing LonMark Functional Profile: Node Object (SFPTnodeObject). Report Node status in response to ObjRequest. It is not used as a heartbeat. |
| TimeStamp | COMM | Domain Broadcast | 7 | SNVT_time_stamp | 3600 | 3600 | Time broadcast to all subsystems for clock synchronization. Shall also be sent when commanded by CDT. |
| FileReq | CDT | All except CDT | 12 | SNVT_file_req | Async | Async | File transfer support variable. |
| FileStat | All | CDT | 27 | SNVT_file_status | Async | Async | File transfer support variable. |
| StatusBits | All except CDT | CDT | 8 | SNVT_state_64 | 9 | 2 | Array of fault and operational status bits to be sent to CDT for display and logging. |
| StatusBits2 | Some nodes other than CDT | CDT | 8 | SNVT_state_64 | 9 | 2 | Supplementary fault and operational status bits. Used by a node that has more boolean data than can fit into the primary StatusBits variable. |
| Parameter | All except CDT | CDT | 3 | UNVT_parameter | 0 (off) | Configur-able. Generally 2 sec. | Numeric values to be sent to CDT for display and logging. |
| Subscribe | CDT | All except CDT | 3 | UNVT_subscribe | Async | Async | Used by the CDT to set StatusBits and Parameter transmission rates. |
| CDTParameter | CDT | Any except CDT | 3 | UNVT_parameter | 0 (off) | Generally 2 sec. | Numeric values to be sent from the CDT to other systems for internal use. |
| CDTStatusBits | CDT | Any except CDT | 3 | SNVT_state_64 | Generally 2 sec. | Generally 2 sec. | Status information to be sent from the CDT to other systems. |
| <Explicit Message> | | | Section 7.2 | <Raw data> | Async | Async | Used to send file contents during transfer. |

| Name | Source Node | Destination Node | Size (bytes) | Type | Default Period (sec) | Normal Period (sec) | Notes |
|---|---|---|---|---|---|---|---|
| NetConfig | N/A | N/A | 1 | SCPTnwrkConfig | N/A | N/A | Configuration property used to support future option for external network management tool. |
| TransferState | CDT | Domain Broadcast | 2 | SNVT_state | Async | Async | Used to indicate that normal data exchanges should be suspended when file transfer is in progress. |
| SCIUpdateReq | CDT | All except CDT | 28 | UNVT_sci_update_request | Async | Async | Used to administer the subsystem SCI update process including file transfer and installation. |
| SCIUpdateResp | All except CDT | CDT | 2 | UNVT_sci_update_response | Async | Async | Response to indicate subsystem state during SCI update. |
| SCIVersionReq | CDT | All except CDT | 2 | UNVT_sci_version_request | Async | Async | Used to request SCI version information from the subsystem. |
| SCIVersion | All except CDT | CDT | 31 | UNVT_sci_version | Async | Async | Used to transfer SCI version information to the CDT. Information for one SCI is transferred at a time. |
| Additional Variables | Any except CDT | Any except CDT | | | | | A limited amount of data may be exchanged between nodes, if required, subject to approval by the network integrator. |

### 6.3.3 Network Variable Services

The LonTalk protocol provides for various types of services when transporting messages across the network. The types of services used depend on the reliability required, bandwidth utilization, number of destination nodes, etc.

The following table shows the service types used within the Silverliner V car monitoring network design.

**Table 6: Network Variable Service Configuration**

| Network Variable | Service | Address Type |
|---|---|---|
| ObjRequest<br>ObjStatus<br>StatusBits<br>StatusBits2<br>Subscribe<br>FileReq<br>FileStat<br>SCIUpdateReq<br>SCIUpdateResp<br>SCIVersionReq<br>SCIVersion | ACKD | Subnet/node |
| Parameter | UNACKD | Subnet/node |
| TimeStamp | UNACKD_RPT | Domain Broadcast |
| TransferState | UNACKD | Domain Broadcast |
| CDTParameter<br>CDTStatusBits | UNACKD | Subnet Broadcast |

Precise details of configuration for each network variable are defined in individual subsystem ICDs, in a section headed “Self-Installation Data”.

## 6.4 Node Object Variables

### 6.4.1 TimeStamp

This network variable is provided to the network by the COMM system based on date and time information from its internal system clock. The TimeStamp network variable is of type SNVT_time_stamp. The COMM system supplies a time stamp update each hour or on command.

### 6.4.2 ObjRequest

The ObjRequest input network variable is a mandatory variable for the Node Object #0 as defined in LonMark Functional Profile: Node Object (SFPTnodeObject). The object ID indicates the functional block index on the device that is the recipient of the request. If the object ID is set to 0 to specify the Node Object as the recipient of the request, the request is intended for the entire node.

The following requests are used in this implementation:

- RQ_NORMAL

- RQ_UPDATE_STATUS
- RQ_REPORT_MASK
- RQ_CLEAR_STATUS

See LonMark Functional Profile: Node Object for further details.

### 6.4.3 ObjStatus

The ObjStatus output network variable is a mandatory variable for the Node Object #0 as defined in LonMark Functional Profile: Node Object (SFPTnodeObject). With the object ID set to 0, the status of the device itself and all functional blocks is reported. The SNVT_obj_status structure defines the status bits available for ObjStatus to report. Table 7 defines the status bits used in this implementation.

**Table 7: Object Status Bit Used**

| STATUS BIT | USED? |
|---|---|
| invalid_id | Yes |
| invalid_request | Yes |
| disabled | No |
| out_of_limits | No |
| open_circuit | No |
| out_of_service | Yes |
| mechanical fault | No |
| feedback_failure | No |
| over_range | No |
| under_range | No |
| electrical_fault | No |
| unable_to_measure | No |
| comm_failure | Yes |
| fail_self_test | No |
| self_test_in_progress | No |
| locked_out | No |
| manual_control | No |
| in_alarm | No |
| in_override | No |
| report_mask | Yes |
| programming_mode | Yes |
| programming_fail | Yes |
| alarm_notify_disabled | No |
| reset_complete | No |

See LonMark Functional Profile: Node Object for further details.

### 6.4.4 FileReq and FileStat (File Transfers)

See section 7.2.

### 6.4.5 NetConfig Configuration Property

The Node Object should implement the Network Configuration Source property. The purpose of implementing this configuration property is to permit future possible installation of the car network using a network management tool, instead of self-installation.

All nodes should have a default value defined in the installed software image and verified as part of the node's manufacturing process; the default value shall be CFG_LOCAL. If the value of the property is reset to CFG_EXTERNAL, it means that a network management tool has taken over the management of the node, and the node must not set its own network configuration, i.e. self-installation is disabled. Operationally, every node must first check the value of the Network Configuration Source property as part of its startup initialization and only modify its configuration tables if the property is set to CFG_LOCAL.

| | |
|---|---|
| Note: | The NetConfig property is recommended for LonMark compliance for all LonWorks nodes but will not be used in the SEPTA car monitoring network. |

## 6.5 Monitoring Object Variables

### 6.5.1 StatusBits and StatusBits2

The StatusBits network variable is provided to transport up to 55 fault or operating status bits to the CDT. Subsystems that require more than 55 status bits shall implement the StatusBits2 network variable. These two variables shall respond identically to all control directives described for StatusBits in this document.

The meaning of each bit shall be determined and documented by the subsystem supplier. This documentation shall be provided to Rotem and Quester Tangent for inclusion into the individual subsystem ICD. The details shall include, but not be limited to: description and cause of event, correlation with subsystem internal event logs, subsystem and train impact, subsystem reaction, recovery etc.

The StatusBits network variable is defined using the standard SNVT_state_64 structure. Bits in this structure are enumerated from 0 to 63 (Bit 0 is reserved); bits within each byte are enumerated from msb (most significant bit) to lsb (least significant bit). For example, bit #9 is the bit in the second byte of the structure corresponding to bitmask 0x40 (hexadecimal 40).

For additional monitoring of network performance, the bit positions 56 to 63 in variable StatusBits shall not be used for a data, but as a counter. The sender shall increment this 8-bit value on each transmission of the StatusBits variable (not including retries within the Neuron chip). The purpose of this counter is to monitor the number of StatusBits network variable updates transmitted but not received.

### 6.5.2 Parameter

The meaning and index of each numeric parameter shall be documented in the individual subsystem ICD.

The Parameter network variable is defined using the following structure:

```
typedef struct {
      unsigned short    param_id;
      signed long       param_value;
} UNVT_parameter;
```

The structure has two members:

- param_id: is an 8-bit unsigned integer . It identifies which parameter is being sent; the valid range is 1 to 255, 0 shall not be used.
- param_value: is a 16-bit value of the parameter being sent. The field's valid range is from 0x0000 to 0x7FFF for positive values and from 0x8001 to 0xFFFF for negative values. The value 0x8000 is reserved to indicate "unknown".

### 6.5.3 Subscribe

The Subscribe network variable is used to control the network traffic as follows:

- Control of the StatusBits and StatusBits2 network variable transmission rate.
- Selection and control of the Parameter network variable transmission rates.

The Subscribe network variable is defined using the following structure:

```
typedef struct {
      unsigned short    param_id;
      unsigned long     update_rate;
} UNVT_subscribe;
```

The structure has two members:

- param_id: is an 8-bit unsigned integer. It identifies the object that is being requested; the valid range is 0 to 255. Value 0 is used to request transmission of the StatusBits network variable (and StatusBits2 where applicable). Values greater than zero (>0) are used to request subsystem-specific Parameter transmissions.
- update_rate: is a 16-bit value that represents the time interval in milliseconds. Valid values are multiples of 500 ms over a range from 0 to 60000 ms. The value of zero (0) indicates transmission is stopped.

After CDT node startup or communication loss with any subsystem (see section 6.2.3.2), the CDT node awaits the receipt of the first StatusBits network variable from the subsystem. Then the CDT sends its Subscribe network variable multiple times in the following sequence:

1. StatusBits transmission rate setting (param_id = 0).
2. First Parameter selection (param_id > 0) and its transmission rate setting.
3. Other Parameter selections and their transmission rate settings.

If the update_rate equals zero (0) then the subsystem shall turn off periodic updates for the parameter (with the given param_id). If the update_rate for the parameter is already zero (0) then the subsystem shall send the parameter once when Subscribe is received.

### 6.5.4 CDTParameter

The CDTParameter network variable is used to pass quantitative data from the CDT to other subsystems. Parameters required for proper vehicle operation and monitoring have been determined by Quester Tangent following consultations with subsystem suppliers and are documented by Quester Tangent in individual subsystem ICDs.

The format and usage of the CDTParameter variable is the same as the Parameter variable described in section 6.5.2.

Upon CDT initialization, the CDT transmits the CDTParameter network variable no less frequently than the transmission rates documented in the individual subsystem ICDs.

### 6.5.5 CDTStatusBits

The CDTStatusBits network variable is used to pass boolean data from the CDT to other subsystems. Status information required for proper vehicle operation and monitoring has been determined by Quester Tangent following consultations with subsystem suppliers and is documented by Quester Tangent in individual subsystem ICDs.

The format and usage of the CDTStatusBits variable is the same as the StatusBits variable described in section 6.5.1.

Upon CDT initialization, the CDT transmits the CDTStatusBits network variable no less frequently than the transmission rate documented in the individual subsystem ICDs.

### 6.5.6 TransferState

The purpose of the TransferState is to instruct all subsystems to stop transmitting network variables. The TransferState message is of type SNVT_state and is broadcast unacknowledged to all other subsystems on the network.

| | |
|---|---|
| Note: | This variable is intended for use during dedicated file transfers such as an SCI update. |

At the start of the file transfer, the CDT shall transmit TransferState with SNVT_state.bit0 = 1. Also, every 10 seconds (nominal) following this for the duration of the file transfer, the CDT shall transmit TransferState with SNVT_state.bit0 =1. After the file transfer is complete, the CDT shall transmit TransferState once with SNVT_state.bit0 = 0.

All nodes shall not transmit any network messages following the receipt of TransferState with SNVT_state.bit0 = 1. These nodes shall resume network communication at the transmission rates previously set when a TransferState message with SNVT_state.bit0 = 0 is received, or 60 seconds following receipt of the last TransferState message with SNVT_state.bit0 = 1. No loss of communication (see section 6.2.3.2) shall be reported in these circumstances.

## 6.6 SCI Update Object Variables

### 6.6.1 SCIUpdateReq

The CDT uses the SCIUpdateReq variable to notify the subsystem of a SCI update. This network variable is defined using the following structure:

```
typedef struct {
      unsigned short    request;
      unsigned short    sci_index;
      unsigned char     sci_name[9];
      unsigned char     sci_ver[9];
      unsigned quad     checkvalue;
      unsigned quad     filesize;
} UNVT_sci_update_init;
```

| | |
|---|---|
| Note: | Quester Tangent may revise the structure definition based on feedback regarding the specific requests from subsystem suppliers. |

The following structure members are required:

- request: The requests are defined in Table 8.
- sci_index: The index for the specific subsystem SCI to be updated. If only one SCI exists on the subsystem, the index shall be set to 1. A value of 0 shall not be used.
- sci_name: The SCI acronym identifier up to 8 ASCII characters in length with a terminating null character.

The following structure members are optional:

- sci_ver: The SCI version identifier up to 8 ASCII characters in length with a terminating null character.
- checkvalue: The SCI image file check value for verifying the successful file transfer. The check value is provided by the subsystem supplier and used by the target subsystem. The

check value shall be no more than 32 bits. Typically this value is a CRC checksum over the image.

- filesize: The SCI image file size in bytes. The file size is provided by the subsystem supplier and used by the target subsystem.

**Table 8: Definition of SCI Update Request Values**

| REQUEST VALUE | REQUEST DEFINITION |
|---|---|
| 0 | Invalid |
| 1 | Initiate – The CDT requests the subsystem to prepare for SCI update. |
| 2 | Install – The CDT has completed SCI image file transfer and commands the subsystem to install the update. |
| 3 | Install and Reboot – The CDT has completed SCI image file transfer and commands the subsystem to install the update. The system requires reboot when the installation is complete. |
| 4 – 255 | Invalid |

See section 7.3 for more details of network variable use.

### 6.6.2 SCIUpdateResp

Subsystems use the SCIUpdateResp variable to signal whether an SCI update request can be serviced. The subsystem may refuse the SCI update if it determines an SCI identifier (index, name or revision) is invalid or it cannot perform the update for operational reasons.

The SCIUpdateResp network variable is defined using the following structure:

```
typedef struct {
      unsigned short    response;
      unsigned short    reason;
} UNVT_sci_update_response;
```

The responses and reason codes are defined in the following tables.

**Table 9: Definition of SCI Update Response Values**

| RESPONSE VALUE | RESPONSE DEFINITION |
|---|---|
| 0 | Invalid. |
| 1 | Initiate OK: Subsystem is ready for SCI update file transfer. |
| 2 | Initiate Refused: Subsystem refuses the SCI update request. |
| 3 | Install OK: Subsystem has verified the SCI image transfer was successful and the image is a valid update. Installation will proceed. |
| 4 | Install Refused: Subsystem verification of the SCI image transfer failed or the image is not a valid update. Subsystem shall discard the transferred image. |
| 5 – 255 | Invalid. |

**Table 10: Definition of SCI Update Reason Values**

| REASON VALUE | REASON DEFINITION |
|---|---|
| 0 | Invalid. |
| 1 | Insufficient memory |
| 2 | CRC check failure |
| 3 | Active faults |
| 4 | Vehicle in motion |
| 5 | Incompatible version |
| 6 | Propulsion contactors (K, AK) not closed |
| 7 – 255 | Invalid. |

Note: Quester Tangent may define additional values based on feedback regarding the specific requests from subsystem suppliers.

See section 7.3 for more details of network variable use.

## 6.7 SCI Version Object Variables

### 6.7.1 SCIVersionReq

The CDT uses the SCIVersionReq variable to retrieve SCI version information from the subsystem. This network variable is defined using the following structure:

```
typedef struct {
      unsigned short    sci_index;
} UNVT_sci_version_request;
```

The member "sci_index" identifies which SCI version information is requested.

Note: SCI's are numbered starting at 1. Index numbers are defined in individual subsystem ICDs.

### 6.7.2 SCIVersion

The subsystems use the SCIVersion variable to transfer SCI version information to the CDT. Information is available for one SCI at a time.

The SCIVersion network variable is defined using the following structure:

```
typedef struct {
      unsigned short    sci_index;
      unsigned short    sci_type;
      unsigned char     sci_name[9];
      unsigned char     sci_version[9];
      unsigned char     build_date[11];
} UNVT_sci_version;
```

Note: All char arrays shall be null terminated.

The following structure member is required:

- sci_index: The software image index specified as a single integer in the range 1 to 15.
- sci_type: The software type as defined in Table 11.

- sci_name: The software image name specified as a string of 1 to 8 characters in the format defined by the subsystem supplier.
- sci_version: The software image version number specified as a string of 1 to 8 characters in the format defined by the subsystem supplier.

The following structure members are optional:

- build_date: The software image build date specified as a string of 10 characters in the format YYYY-MM-DD where YYYY is the 4-digit year, MM is the 2-digit month (January = 01), DD is the 2-digit day of the month.

**Table 11: Definition of SCI_Type Values**

| SCI_TYPE VALUE | SCI_TYPE DEFINITION |
|---|---|
| 0 | Invalid. |
| 1 | Software: SCI can be updated via the network from the CDT. |
| 2 | Firmware: SCI requires subsystem specific operation to update. This may include physical component replacement or direct physical access to the subsystem. |
| 3 | Neuron: Neuron based SCI. Requires update through the standard functionality provided by the LonTalk protocol management functions. |
| 4 | Operating System: Version information provided for reference purposes only. |
| 5 | Bootloader: Version information provided for reference purposes only. |
| 6 | SCI does not exist: Indicates CDT requested version information using an invalid SCI index. |
| 7 – 255 | Invalid. |

# 7 File Transfers

## 7.1 File Types

File transfers are used for the operations described in the following table:

**Table 12: File Types Supported in File Transfer**

| FILE ID | FILE NAME | DESCRIPTION | SENDER | RECEIVER | INITIATOR |
|---|---|---|---|---|---|
| 257 | Software Image | Binary software image for a subsystem. | CDT | All except CDT | CDT |

Note: See individual subsystem Car Monitoring Network ICDs for details regarding additional file transfers.

## 7.2 File Transfer Protocol

The required implementation of the protocol is a subset of the Echelon File Transfer Protocol described in "File Transfer LonWorks Engineering Bulletin November 1996". The LonWorks file transfer protocol has three (3) logical participants: the Initiator, the Sender and the Receiver as shown in Figure 4. The Sender or Receiver may also act as the Initiator. The CDT is generally the file transfer Initiator. Depending on the file type, the CDT is also the file Sender or file Receiver. Table 12 defines the Initiator, Sender and Receiver for each of the file transfers supported.

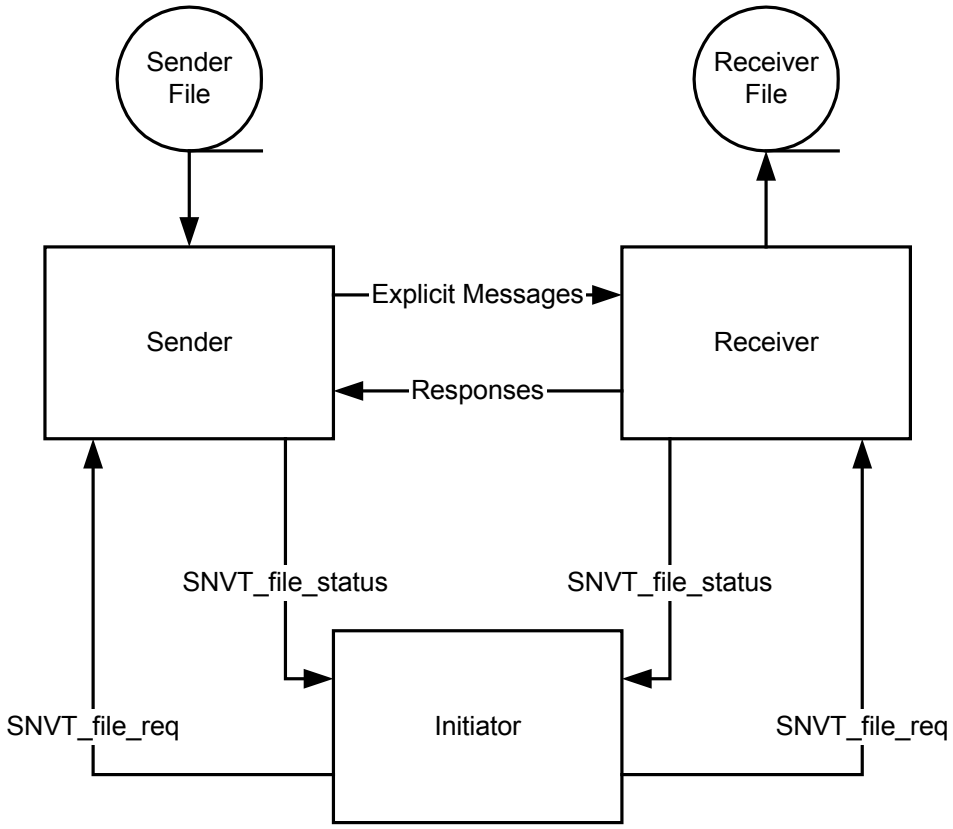



**Figure 4: Logical File Transfer Mechanism**

Two standard network variable types are used to set up the actual file transfer: SNVT_file_request and SNVT_file_status. The data transfer is implemented with explicit messages using a windowed transfer protocol. For detailed information and implementation notes refer to Echelon's File Transfer engineering bulletin. An example Neuron C file transfer application is available from Echelon. All file transfers shall be full file transfers, i.e. from start to end of the available data. There is no requirement for the random access protocol for file transfers described in the Echelon documentation to be supported.

The following parameters shall be used for all file transfers unless otherwise stated in the individual subsystem ICD:

- Packet Size = 32 bytes
- Receiver Timeout = 1000 msec
- Sender Timeout = 1000 msec
- Retry Count = 5
- Authentication = Off
- Priority = Off

## 7.3 SCI Update Protocol

Additional signalling between the CDT and subsystem is required to manage the update of an SCI on the target subsystem. The PTU is connected to the CDT for SCI update. The SCI image and image information file (see section 7.3.1 for file details) reside on the PTU. The image information file is sent to the CDT and is used to populate the SCI update request variable sent to the subsystem.

| | |
|---|---|
| Note: | The Neuron processor software image cannot be updated using this method. All Neuron based nodes shall accept software image update through the standard functionality provided by LonTalk protocol management functions. |

Figure 5 is a sequence diagram that shows the use of network variables to prepare for SCI image transfer and installation. The CDT sends the initial SCI update request variable to the subsystem. If the subsystem is unable to perform an update, the subsystem shall signal the CDT and the update will be aborted. The subsystem shall signal its programming mode and programming success or failure through the ObjStatus network variable (in its Node Object).

The subsystem responds to the update request when it has verified the image information and is prepared for SCI update. The CDT initiates a LonWorks file transfer as described in section 7.2 and sends the file from the PTU to the subsystem.

When the file transfer is complete, the CDT sends an install request to the subsystem. Depending on the subsystem the install request may include a reboot request. The subsystem verifies that the SCI file transfer was completed without error and the SCI is intact. The subsystem responds that the file transfer was successful and the SCI will be installed. If the transfer was incomplete or the image is not suitable for the subsystem, the subsystem shall indicate installation failure, discard the image and resume normal operation.

The CDT monitors the network to determine when the subsystem resumes operation after installation or installation and reboot. Once online, the CDT queries the subsystem for the SCI version information to confirm installation of the SCI update.

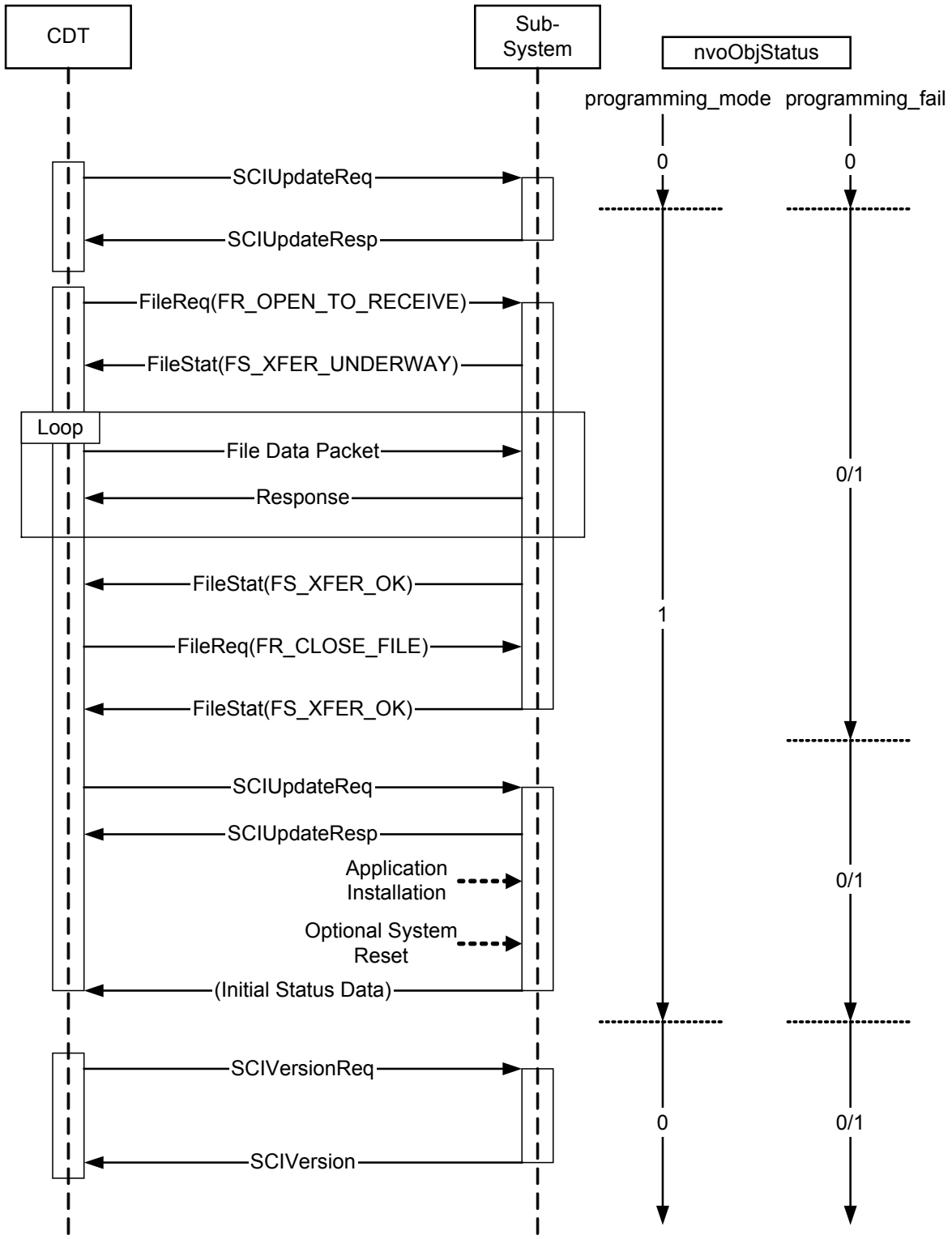



**Figure 5: SCI Update File Transfer Protocol**

### 7.3.1 Software Image File Formats

This file is transferred transparently from the PTU laptop through the CDT to the subsystem that is to be updated. The CDT does not validate or perform any interpretation of the file contents.

Some software image verification information can be transmitted to the subsystem embedded in the SCIUpdateReq network variable when the file transfer is initiated. Not all subsystems will require this functionality. The PTU sends the image information file contents to the CDT separately from the SCI image. The CDT embeds any required information in the SCIUpdateReq network variable when the SCI update initialization request is sent to the subsystem. The software image information file is a comma separated ASCII text file with the following fields on a single line:

```
Subsystem name,SCI Index,SCI name,SCI version,Image checkvalue,Image size,Reboot?
```

- Subsystem name: See Table 13.
- SCI Index: Index for the particular SCI.
- SCI name: up to 8 alphanumeric characters
- SCI version: up to 8 alphanumeric characters
- Image checkvalue: SCI image file verification code (up to 32 bit Hex)
- Image size: SCI image file length in bytes defined as a Hex value
- Reboot?: ‘Y’ or ‘N’ to indicate whether reboot after install is required

Example file:

```
FBCU,2,FBCU2,2E,5FC2447A,13C54,N<CR><LF>
```

**Table 13: Subsystem Names**

| Subsystem | Name |
|---|---|
| Propulsion | PROP |
| Brakes | FBCU |
| Auxiliary Power | APU |
| HVAC | HVAC |
| Communications | COMM |
| ATC | ATC |
| Door System | DCU, DCRP |

Note: All items in the software image file are optional. Omitted items are indicated by back-to-back commas, e.g. "FBCU,2,FBCU2,2E,,,Y"

# 8 Interface Rules and Conventions

## 8.1 Node Documentation

Subsystem LonWorks nodes shall incorporate a maximum of self-documentation information. Mandatory documentation strings include, but are not limited to:

- Node self-documentation string
- Network variable self-documentation strings
- Non-Standard ASCII Program ID string
- Node location string

### 8.1.1 Neuron-based Nodes

The following compiler directive shall be part of the node's source code in order to allow browsing on meaningful network variables names:

```
#pragma enable_sd_nv_names
```

The node self-documentation string shall be implemented as part of the Neuron C source code using the following compiler directive:

```
#pragma set_node_sd_string "node name, supplier name, part number, Neuron SCI revision date"
```

Example for the CDT supplier:

```
#pragma set_node_sd_string "CDT, Quester Tangent, QFG-MS10-0001, 2007-01-08"
```

The Neuron C compiler (in NodeBuilder 3 and later versions) automatically generates a portion of the self-documentation string that documents the functional blocks that are implemented by the application. See the Neuron C Programmer's Guide and Neuron C Reference Guide for details.

Subsystem suppliers shall also provide to Rotem and Quester Tangent the name and order of declaration of each network variable used in the supplier's Neuron C source code. Subsequent modification to this list shall also be transmitted to Rotem and Quester Tangent.

### 8.1.2 Host-based Nodes

| | |
|---|---|
| Note: | Host-based nodes are those that use Echelon's MIP technology. Unless the subsystem demonstrates a valuable reason for using the MIP, use of the MIP as a network interface is discouraged because of its complexity. The MIP interface on the Neuron microcontroller filters and handles certain management messages; other management messages are routed to the host processor. |

All Host-based nodes shall provide functionality to correctly respond to queries by network management tools for self-identification and self-documentation.

The following rules shall be followed to guarantee system reliability:

- Host-based nodes must preserve network variable configuration table contents across hardware resets and power cycles.
- Care shall be taken when implementing the Host and Neuron reset configuration. The Neuron shall not be able to reset the Host, but the Host shall be able to reset the Neuron.
- If network variable selection is performed on the host, then the host application must manage a network variable configuration table, and respond correctly to all the network

management messages sent to it, except “Update Net Variable Config”. A description of the management messages sent to a host can be found in the LonWorks Host Application Programmer’s Guide.

- The host application must respond to network variable fetch and network variable poll request messages with the appropriate data from the request network variable. If the application is in the off-line mode when a network variable poll is received, the response must contain no data. Otherwise, the response must contain the correct number of bytes of data as specified by the network variable type.
- The host application must ensure that the PROGRAM ID and LOCATION ID are present in the network interface.

Subsystem suppliers shall provide to Rotem and Quester Tangent the name and order of declaration of each network variable, as presented in response to network management queries. Subsequent modification to this list shall also be transmitted to Rotem and Quester Tangent.

## 8.2 Neuron Conventions

The Neuron processor is “big-endian”, which means that multi-byte variables are stored with the most significant byte at the lowest memory address.

In Neuron C, data types “int” and “short” are 8-bit values, “long” is a 16-bit value and "quad" is a 32-bit value.

## 8.3 Variable Declaration Rules

### 8.3.1 Neuron-based Nodes

Network variables and configuration properties shall be declared in the order defined in the individual subsystem ICD.

### 8.3.2 Host-based Nodes

Responses to all types of network variable queries (by an external network management or diagnostic tool) shall return data according to the network variable numbering scheme defined in the individual subsystem ICD.

## 8.4 Network Installation

All nodes shall support the Network Management and Diagnostic Services as expressed in Echelon’s Neuron Chip Data Book, Appendix B.

The following rules shall be followed to ease system integration:

- The subsystem shall support the maximum number of 15 address table entries.
- The subsystem shall support two (2) domains.
- All of a node’s application software must be independent of its network configuration.
- The subsystem shall provide external access to its service pin with visual access to the yellow service LED.

### 8.4.1 Self Installation

Individual subsystem ICDs contain self-installation data definitions and details. On start-up, the subsystem shall query its network configuration tables and update them if required, to ensure that settings correspond to the installation location (see section 5.3). The subsystem shall maintain a non-volatile variable to indicate whether self-installation has been performed. The subsystem shall not update configuration tables if the installation location has not changed, to avoid the associated start-up delay.

The following example pseudo-code fragment illustrates a suitable algorithm for self-installation checks. This example is suitable for a Neuron-based node.

```
//
// *** Install node based on location ***
//
void self_install(void)
{
    if (ConfigSrc != CFG_LOCAL)
    }
        if(ee_config_status != NOT_LOCAL_CONFIG)
        }
            ee_config_status = NOT_LOCAL_CONFIG;
        }
    }
    else
    }
        // Fetch local copy of Neuron configuration table
        config_copy = config_data;

        // Check if not yet configured or location has changed
        if ((ee_config_status != LOCAL_CONFIG_DONE) ||
            ( /* <<< config_copy.location != (install_loc) >>> */ ))
        {
            // rebuild configuration table
            // rebuild domain table
            // rebuild address table
            // rebuild network variable table

            // update status to indicate self-installation is complete
            ee_config_status = LOCAL_CONFIG_DONE;
        }
    }
}
```

### 8.4.2 Neuron Chip Firmware Version

The 3150 Neuron chip does not come with firmware. The subsystem supplier shall load the firmware version. All 3150 Neuron chip users are recommended to use the latest available version of firmware.

## 8.5 External Interface File

Each subsystem supplier shall provide Rotem and Quester Tangent with an up-to-date External Interface File (XIF) for the subsystem. The XIF documents the subsystem's network variables, messages tags and hardware related parameters, including the Neuron chip type, the clock rate, the bit rate, and the medium type. The file format consists of a number of records including some global values for use by the network management tool. The global values include the following information:

- The subsystem standard program ID.
- The maximum number of domains to which the node can belong.
- The number of address table slots.
- Whether the node handles incoming explicit messages.

- The number of network variables defined.
- The number of message tags defines.
- The subsystem self-documentation string.
- The self-documentation strings for each network variable declared on the subsystem.

For Neuron based subsystems, the compilation tools automatically create the interface and documentation.

For host-based subsystems, the NodeUtil utility (version 2.04 or later) can then be used to generate an XIF if the support functionality has been implemented by the host application.

This interface file shall be delivered with every update of the subsystem network interface.

## 8.6 Additional Features

Further details will be added as this document evolves, following consultation with subsystem suppliers.

# 9 Subsystem Commissioning

## 9.1 Subsystem Interface Definition Process

As the network integrator for the Silverliner V car monitoring network, Quester Tangent develops subsystem interface definitions using the following strategy:

- QT provides an initial release of the current document to characterize the methodology used to interface to the CDT.
- QT provides a template for subsystem-specific interface definitions.
- Subsystem suppliers fill out the template with as much information as possible.
- QT evaluates the filled-out template and requests additional information as required.
- QT integrates the supplier information into a subsystem-specific Interface Control Document (ICD). This document becomes a statement of interface requirements for the subsystem.
- QT updates this top-level ICD and subsystem-specific ICDs to provide final interface details such as network variable selectors used for self-installation.

Rotem will manage interactions between QT and subsystem suppliers. It is hoped that direct interactions between QT and suppliers will be possible to promote rapid convergence on an agreed interface.

It is anticipated that there will be some degree of iterative refinement of interface definitions as the development of the car and the definition of its monitoring requirements evolves. QT will take steps to finalize essential details of interface definitions as early as possible to allow suppliers to finalize their own designs. However, suppliers must anticipate and be flexible to the evolutionary nature of this process.

## 9.2 Supplier In-house Testing

Each subsystem supplier shall perform qualification (type) tests on a production unit, or on an equivalent prototype. Subsystem qualification shall include the complete EMC/EMI testing of the network interface and the complete node functionality according to this specification. Testing shall include valid parameter and status event simulation using the complete subsystem, not the network interface in isolation.

Quester Tangent provides a test requirements list that the supplier can use to verify that the subsystem test procedure covers the scope of the network interface design defined in this document.

Quester Tangent reviews test procedures prior to the supplier's in-house testing, and reviews test results prior to interoperability testing.

In order to perform qualification tests, the subsystem supplier shall emulate the role of the CDT in network communications using suitable test tools. For example, a custom application running on COTS hardware, plus COTS tools such as NodeUtil and Protocol Analyzer would be suitable. The emulation shall test all aspects of the subsystem interface with the CDT, including, but not limited to, requesting status and parameter updates at different transmission rates.

The subsystem supplier shall demonstrate successful in-house testing of the subsystem prior to interoperability testing.

## 9.3 Interoperability Testing

A communication test shall be performed between the CDT and each subsystem node before the equipment is integrated into the vehicle. Unless otherwise agreed, this test will be performed by Quester Tangent at Quester Tangent's facilities with the assistance of the subsystem supplier.

The subsystem presented for testing shall exhibit its full car network functionality. If it is not practical to perform network testing with the complete physical subsystem, the subsystem supplier shall provide a means to simulate valid parameter and status data as would be generated by the complete physical subsystem through its network interface to the CDT.

Quester Tangent may perform supplementary conducted emissions and noise susceptibility testing of a subsystem node as part of the interoperability testing.

## 9.4 Vehicle Integration Testing

Each subsystem supplier shall provide support for network functional tests during vehicle integration testing at Rotem and SEPTA facilities.

Quester Tangent shall establish baseline network characteristics during this phase.